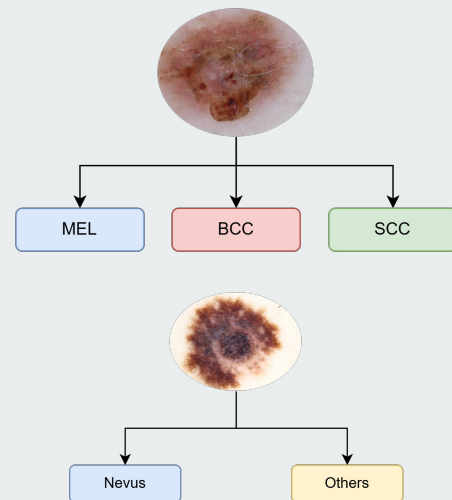


Project 2: Deep Learning for Skin Lesion Classification.

Muhammad Kabir Hamzah
Faisal Farhan

Erasmus Mundus Joint Masters in Medical Imaging and Applications



Introduction



Objective: Develop a computer aided algorithm for the diagnosis in dermoscopic images.

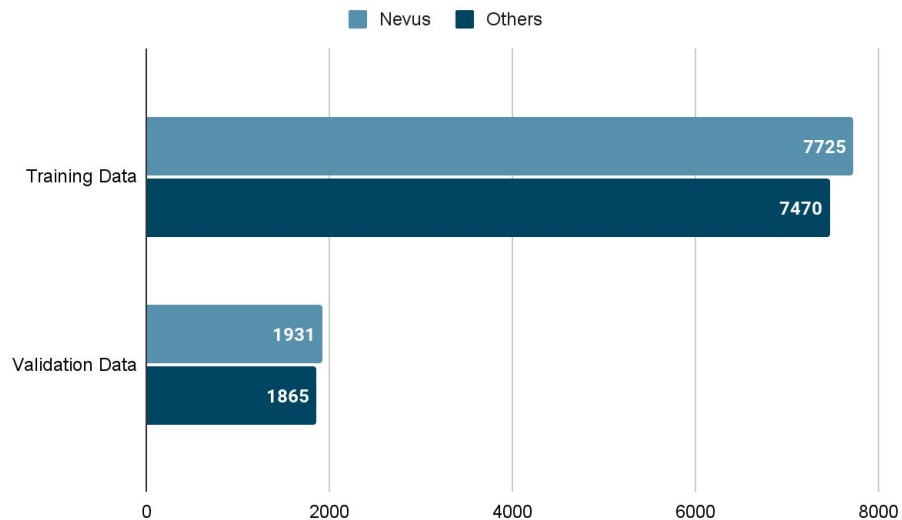
Challenges:

#1. The binary problem of classifying Nevus images vs all the others.

#2. A three-class problem consisting on the classification of cancers: melanoma vs basal cell carcinoma vs squamous cell carcinoma.

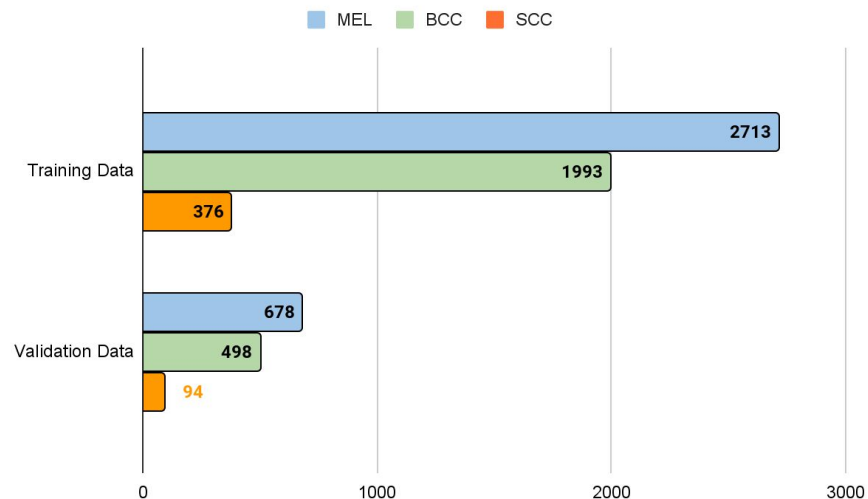
Dataset

#Challenge 1



Total Training Data: 15195
Total Validation Data: 3796
Total Test Data: 6340

#Challenge 2



Total Training Data: 5760
Total Validation Data: 1270
Total Test Data: 2121

Nomenclature

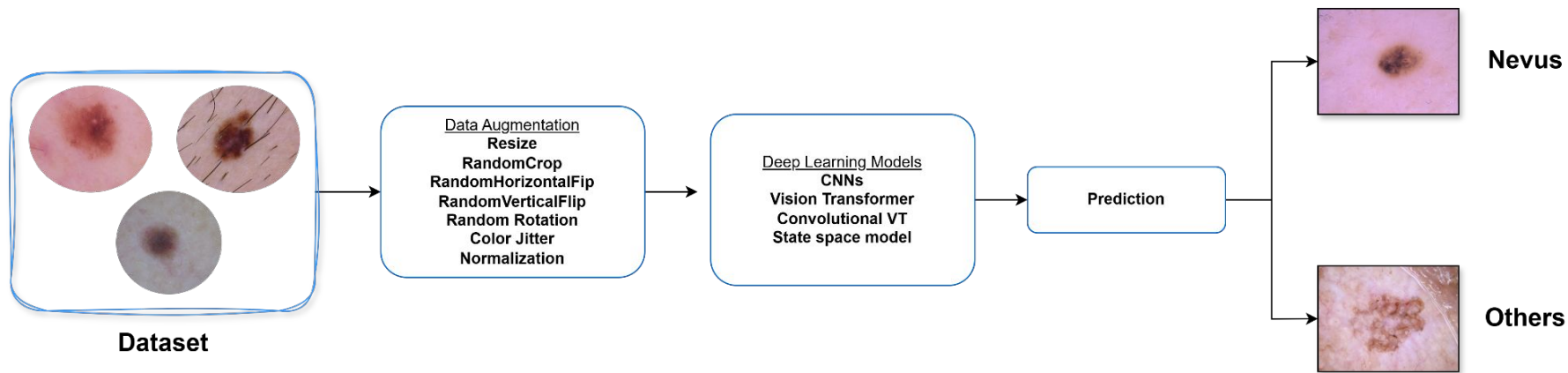


- Train dataset -> Provided Train dataset
- Validation dataset -> Provided Validation dataset
- Test dataset -> Provided Test dataset
- Beta_Train dataset -> Random subset of Train dataset(80%)
- Beta_Val dataset -> Random subset of Train dataset (20%)

Challenge 1

Binary Classification Problem

Challenge 1: #Classification Pipeline



Challenge 1: #Pre-processing & Augmentation

Resize	256 x 256 x 3
RandomCrop	224 x 224 x 3
RandomHorizontalFlip	
RandomVerticalFlip	
RandomRotation	
ColorJitter	
Normalize using computed statistics	Mean = [0.6679, 0.5354, 0.5194] std = [0.2224, 0.2053, 0.2171]

Deep Learning Models



CNNs

MobileNet
DenseNet
Efficient Net
ResNext
ConvNext

Transformers

Vision
Transformer (base)
Data efficient
transformer (Deit)

CNN+Transformer

Convolutional
Vision
Transformer
(ConVit)

State Space Model

Medical Mamba
(MEDMAMBA)

#Experiment-1

Transfer Learning

We started off by selecting pretrained models with IMAGENET weights and evaluating them on the **Validation dataset**.

Preprocessing steps:

- **Resize:** 256 x 256 x 3
- **CenterCrop:** 224 x 224 x 3
- **Normalize using Image Net statistics:** mean = [0.485, 0.456, 0.406] ; std = [0.229, 0.224, 0.225]

EXPERIMENTS

#Experiment-1

Transfer Learning

Experimental RESULTS

Model	Val accuracy	Val Kappa
Mobilenetv2	0.46	-0.088
DenseNet	0.48	-0.027
resnext50	0.48	-0.023
efficientnetv2_l	0.52	0.035
convnextv2_base	0.45	-0.084
convit_small	0.53	0.056
deit_base_patch16_224	0.49	-0.009
vitbase16	0.58	0.164

#Experiment-2

3 Layer Fine Tuning

The second experiment was fine tuning, and we only trained the last 3 Layers of the networks on Beta_Train dataset and Beta_Val dataset and tested on **Validation dataset**.

Preprocessing and augmentation steps:

- **Resize:** 256 x 256 x 3
- **RandomCrop:** 224 x 224 x 3
- **RandomHorizontalFlip**
- **RandomVerticalFlip**
- **RandomRotation**
- **ColorJitter**
- **Normalize using computed statistics:** mean = [0.6679, 0.5354, 0.5194]; std = [0.2224, 0.2053, 0.2171]

EXPERIMENTS

#Experiment-2

3 Layer Fine Tuning

Experimental RESULTS

Model	Train Accuracy	Train Kappa	Val Accuracy	Val kappa	Test accuracy	Test Kappa
Mobilenetv2	0.8518	0.7033	0.8473	0.6947	0.8303	0.6595
DenseNet	0.8806	0.7611	0.8578	0.7155	0.8546	0.7083
resnext50	0.8806	0.761	0.8588	0.7174	0.8488	0.6967
efficientnetv2_l	0.9172	0.8342	0.8786	0.7570	0.8604	0.7203
convnextv2_base	0.8998	0.7993	0.8875	0.7749	0.8812	0.7622
convit_small	0.8724	0.7445	0.8460	0.692	0.8596	0.7191
deit_base_patch16_224	0.8812	0.7621	0.8667	0.7335	0.8630	0.7257
vitbase16	0.8356	0.6709	0.8384	0.6769	0.8296	0.6584

#Experiment-3

Fine Tuning (80-20)

The third experiment was fine tuning, but this time we trained the entire networks using Beta_Train dataset and Beta_Val dataset and tested on **Validation dataset**.

Preprocessing and augmentation steps:

- **Resize:** 256 x 256 x 3
- **RandomCrop:** 224 x 224 x 3
- **RandomHorizontalFlip**
- **RandomVerticalFlip**
- **RandomRotation**
- **ColorJitter**
- **Normalize using computed statistics:** mean = [0.6679, 0.5354, 0.5194]; std = [0.2224, 0.2053, 0.2171]

EXPERIMENTS

#Experiment-3

Fine Tuning (80-20)

Experimental RESULTS

Model	Train Accuracy	Train Kappa	Val Accuracy	Val Kappa	Test accuracy	Test Kappa
Mobilenetv2	0.9410	0.8820	0.8810	0.7620	0.8772	0.7541
DenseNet	0.9890	0.9780	0.9010	0.8020	0.8930	0.7857
resnext50	0.9860	0.9720	0.8960	0.7910	0.8988	0.7973
efficientnetv2_l	0.9970	0.9950	0.9160	0.8310	0.9041	0.8080
convnextv2_base	0.9980	0.9970	0.9170	0.8340	0.9086	0.8171
convit_small	0.9820	0.9640	0.8940	0.7890	0.8917	0.7833
deit_base_patch16_224	0.9940	0.9870	0.913	0.826	0.8991	0.7980
vitbase16	0.9790	0.9580	0.8960	0.7910	0.7647	0.8825

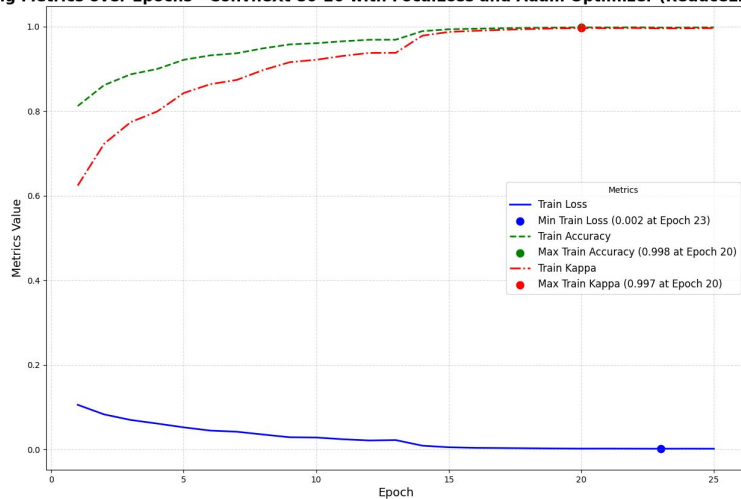
EXPERIMENTS

#Experiment-3

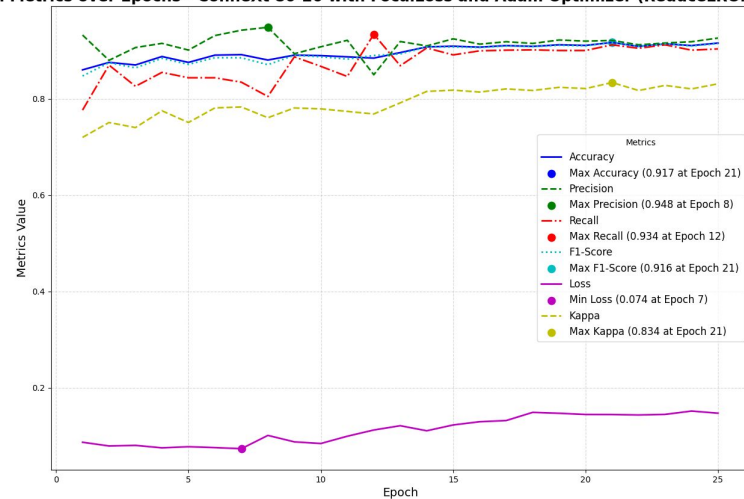
Fine Tuning (80-20)

Experimental RESULTS (CovneXt)

Training Metrics over Epochs - ConvneXt 80-20 with FocalLoss and Adam Optimizer (ReduceLRonPlateau)



Val Metrics over Epochs - ConneXt 80-20 with FocalLoss and Adam Optimizer (ReduceLRonPlateau)



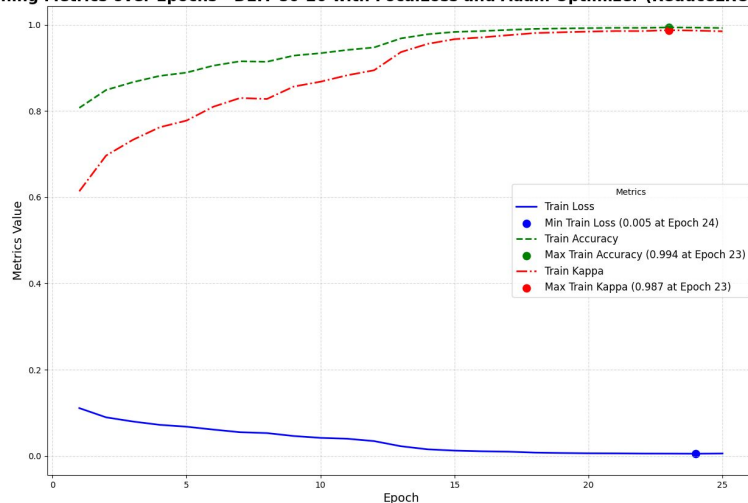
EXPERIMENTS

#Experiment-3

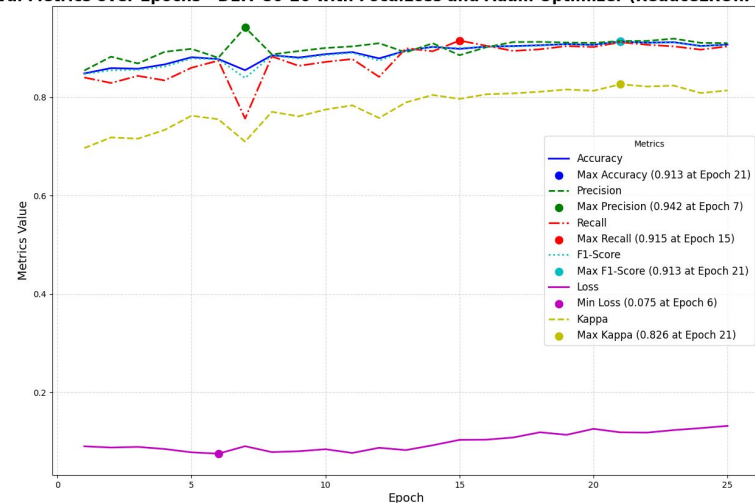
Fine Tuning (80-20)

Experimental RESULTS (Deit)

Training Metrics over Epochs - DEIT 80-20 with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



Val Metrics over Epochs - DEIT 80-20 with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



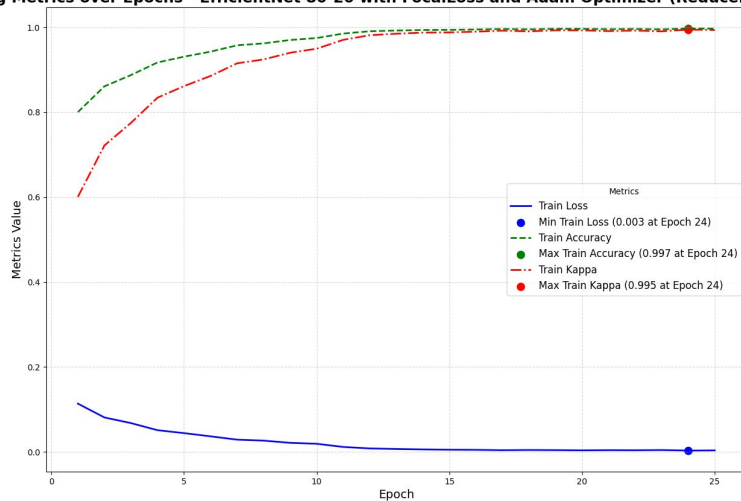
EXPERIMENTS

#Experiment-3

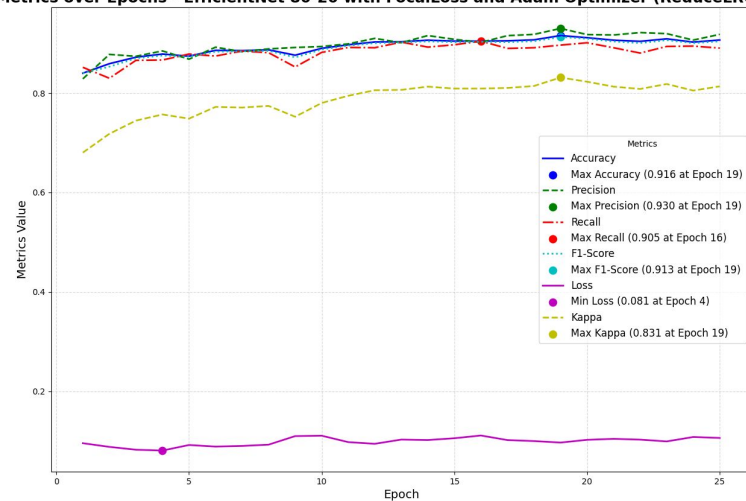
Fine Tuning (80-20)

Experimental RESULTS (EfficientNet)

Training Metrics over Epochs - EfficientNet 80-20 with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



Val Metrics over Epochs - EfficientNet 80-20 with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



EXPERIMENTS

#Experiment-4

Fine Tuning

The fourth experiment was fine tuning, but this time we trained the entire networks using Train dataset and Val dataset and therefore no evaluation. We also added a **Mamba network**.

Preprocessing and augmentation steps:

- **Resize:** 256 x 256 x 3
- **RandomCrop:** 224 x 224 x 3
- **RandomHorizontalFlip**
- **RandomVerticalFlip**
- **RandomRotation**
- **ColorJitter**
- **Normalize using computed statistics:** mean = [0.6679, 0.5354, 0.5194]; std = [0.2224, 0.2053, 0.2171]

EXPERIMENTS

#Experiment-4

Fine Tuning

Experimental RESULTS

Model	Train Accuracy	Train Kappa	Val accuracy	Val Kappa
Mobilenetv2	0.9335	0.8669	0.890	0.779
DenseNet	0.9572	0.9143	0.901	0.802
resnext50	0.9705	0.9409	0.904	0.809
efficientnetv2_l	0.9944	0.9888	0.925	0.849
convnextv2_base	0.998	0.9961	0.927	0.853
convit_small	0.9914	0.9828	0.911	0.822
deit_base_patch16_224	0.9944	0.9888	0.913	0.825
vitbase16	0.9731	0.9461	0.902	0.804
MedMamba	0.7950	0.5650	0.830	0.659

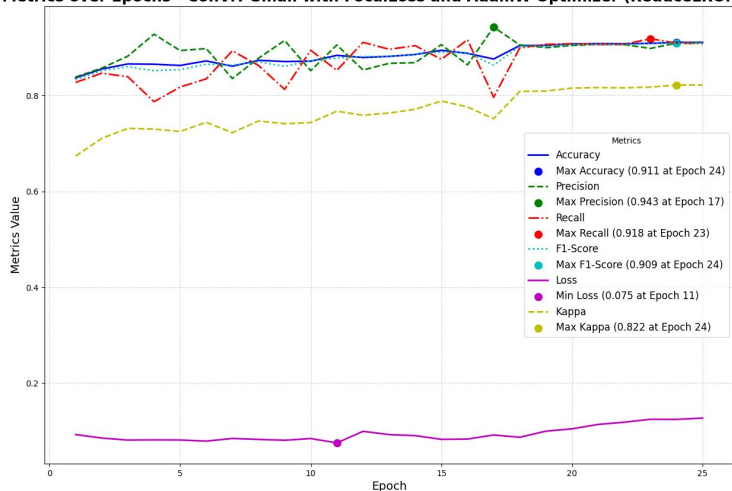
EXPERIMENTS

#Experiment-4

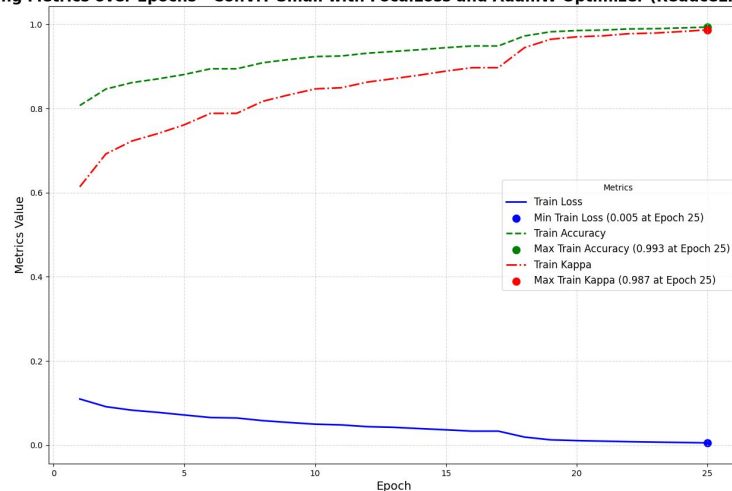
Fine Tuning

Experimental RESULTS (ConViT)

Val Metrics over Epochs - ConViT Small with FocalLoss and AdamW Optimizer (ReduceLRonPlateau)



Training Metrics over Epochs - ConViT Small with FocalLoss and AdamW Optimizer (ReduceLRonPlateau)



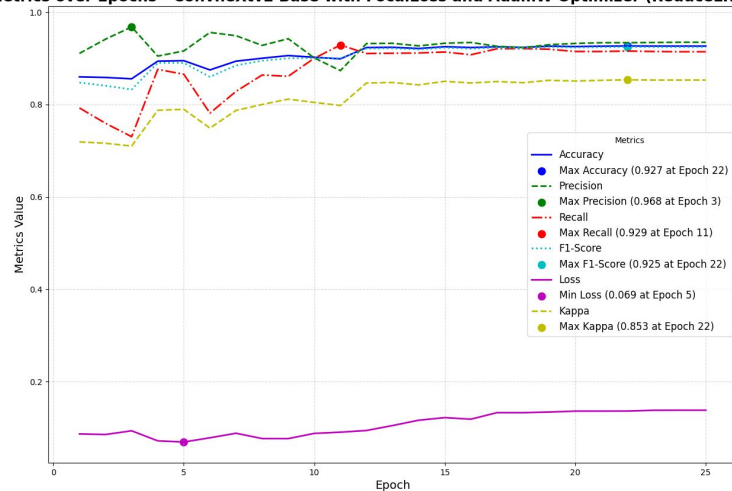
EXPERIMENTS

#Experiment-4

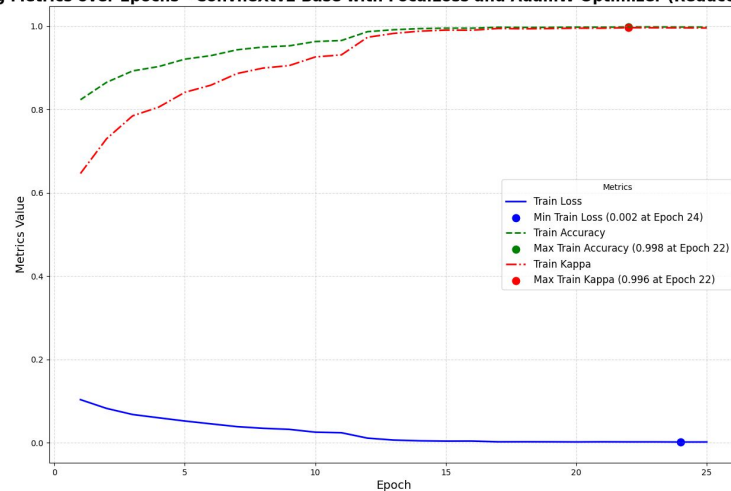
Fine Tuning

Experimental RESULTS (ConvneXt)

Val Metrics over Epochs - ConvneXtv2 Base with FocalLoss and AdamW Optimizer (ReduceLRonPlateau)



Training Metrics over Epochs - ConvneXtv2 Base with FocalLoss and AdamW Optimizer (ReduceLRonPlateau)



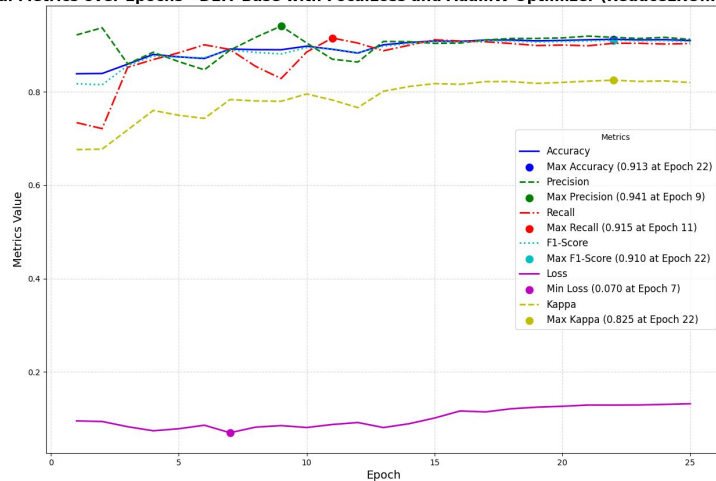
EXPERIMENTS

#Experiment-4

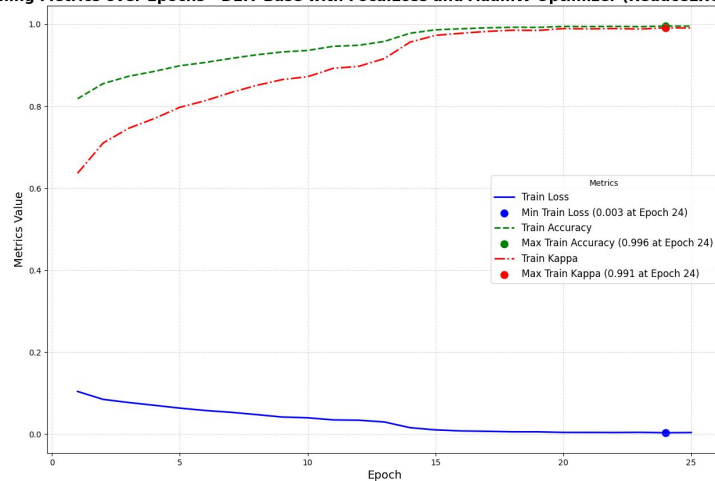
Fine Tuning

Experimental RESULTS (DEiT)

Val Metrics over Epochs - DEiT Base with FocalLoss and AdamW Optimizer (ReduceLROnPlateau)



Training Metrics over Epochs - DEiT Base with FocalLoss and AdamW Optimizer (ReduceLROnPlateau)



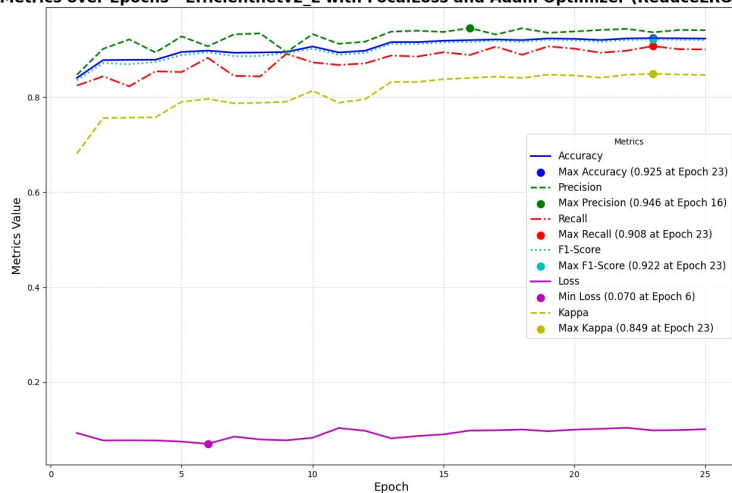
EXPERIMENTS

#Experiment-4

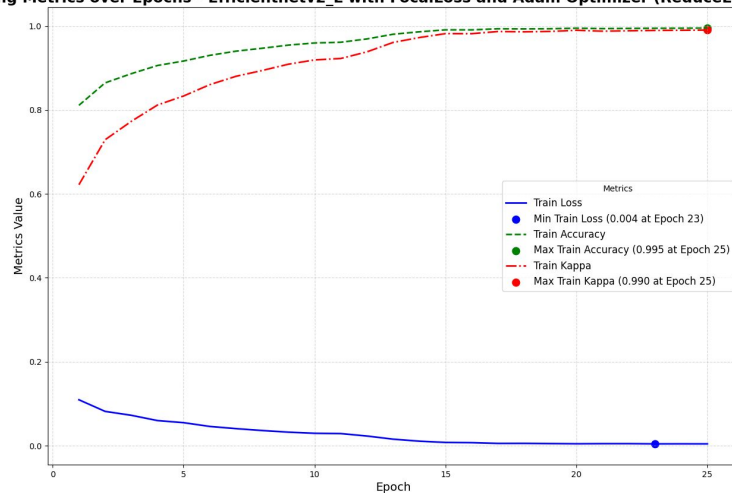
Fine Tuning

Experimental RESULTS (EfficientNet)

Val Metrics over Epochs - Efficientnetv2_L with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



Training Metrics over Epochs - Efficientnetv2_L with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



EXPERIMENTS

#Experiment-5

Ensemble

Experimental RESULTS

Evaluation table challenge 1 (Convit, convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9294	0.8587
Hard	0.9283	0.8565

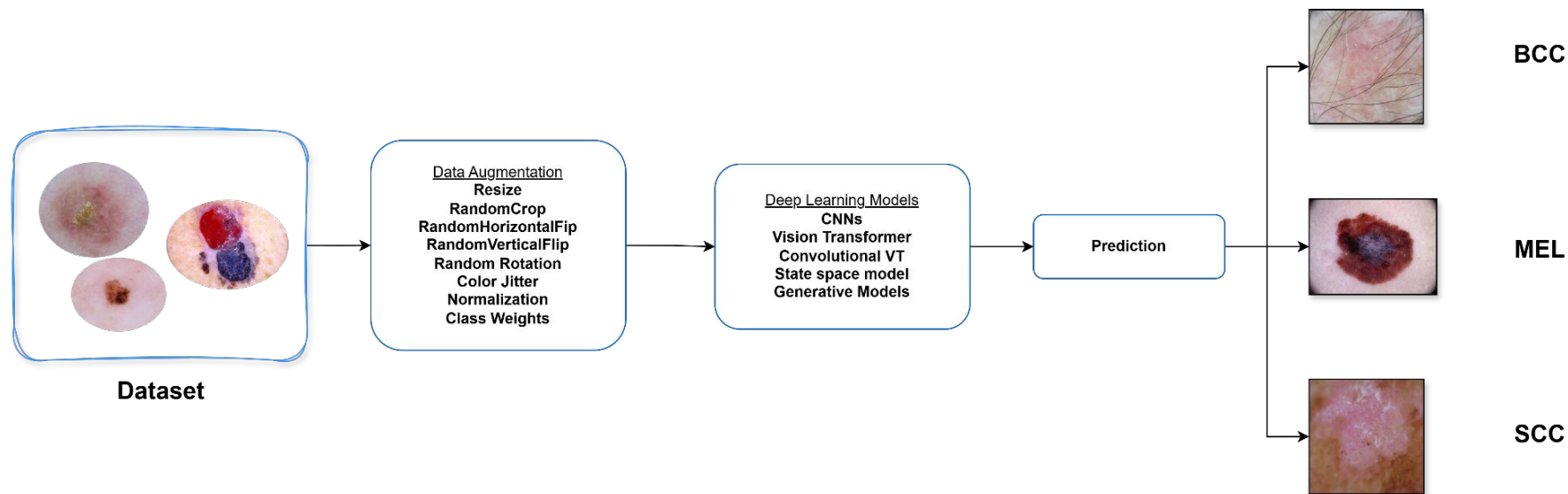
Evaluation table challenge 1 (convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9312	0.8624
Hard	0.9320	0.8640

Challenge 2

Three Class Problem

Classification Pipeline: #Challenge 2



Challenge 2: #Pre-processing & Augmentation

Resize	256 x 256 x 3
RandomCrop	224 x 224 x 3
RandomHorizontalFlip	
RandomVerticalFlip	
RandomRotation	
ColorJitter	
Normalize using computed statistics	Mean = [0.6679, 0.5354, 0.5194] std = [0.2224, 0.2053, 0.2171]
Class Weight	{'bcc': 0.1889, 'mel': 0.1386, 'scc': 0.6725}

Deep Learning Models

CNNs

MobileNet
DenseNet
EfficientNet
ResNext
ConvNext

Transformers

Vision
Transformer (base)
Data efficient
transformer (Deit)

CNN+Transformer

Convolutional
Vision
Transformer
(ConVit)

State Space Model

Medical Mamba
(MEDMAMBA)

Variational Autoencoders (VAE)

Vanilla VAE
Conditional VAE
Categorical VAE
Beta VAE

NOTES



- For Challenge 2 we followed the same five **experiments (1-5)** as Challenge 1.
- Similar preprocessing steps were applied.
- Experiment 6 (Variational Autoencoders) was implemented for Challenge 2.

#Experiment-1

Transfer Learning

We started off by selecting pretrained models with IMAGENET weights and evaluating them on the **Validation dataset**.

Preprocessing steps:

- **Resize:** 256 x 256 x 3
- **CenterCrop:** 224 x 224 x 3
- **Normalize using Image Net statistics:** mean = [0.485, 0.456, 0.406] ; std = [0.229, 0.224, 0.225]

EXPERIMENTS

#Experiment-1

Transfer Learning

Experimental RESULTS

Model	Val accuracy	Val Kappa
Mobilenetv2	0.29	-0.0378
DenseNet	0.31	-0.040
resnext50	0.45	-0.0279
efficientnetv2_l	0.48	-0.032
convnextv2_base	0.32	0.021
convit_small	0.37	-0.0169
deit_base_patch16_224	0.35	-0.063
vitbase16	0.35	-0.037

EXPERIMENTS

#Experiment-4

Fine Tuning

The fourth experiment was fine tuning, but this time we trained the entire networks using Train dataset and Val dataset and therefore no evaluation. We also added a **Mamba network**.

Preprocessing and augmentation steps:

- **Resize:** 256 x 256 x 3
- **RandomCrop:** 224 x 224 x 3
- **RandomHorizontalFlip**
- **RandomVerticalFlip**
- **RandomRotation**
- **ColorJitter**
- **Normalize using computed statistics:** mean = [0.6679, 0.5354, 0.5194]; std = [0.2224, 0.2053, 0.2171]

EXPERIMENTS

#Experiment-4

Fine Tuning

Experimental RESULTS

Model	Train Accuracy	Train Kappa	Val accuracy	Val Kappa
Mobilenetv2	0.8486	0.7412	0.891	0.808
DenseNet	0.8954	0.8173	0.926	0.869
resnext50	0.9227	0.8644*	0.928	0.872
efficientnetv2_l	0.9518	0.9147	0.949	0.909*
convnextv2_base	0.9480	0.908	0.947	0.906
convit_small	0.8600	0.7598	0.917	0.852
deit_base_patch16_224	0.9487	0.9092	0.946	0.904
vitbase16	0.8854	0.8012	0.906	0.835
MedMamba	0.6760	0.7330	0.769	0.594

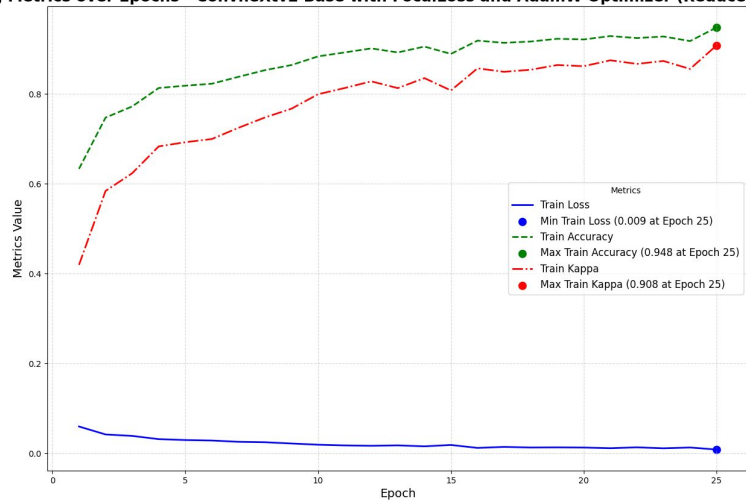
EXPERIMENTS

#Experiment-4

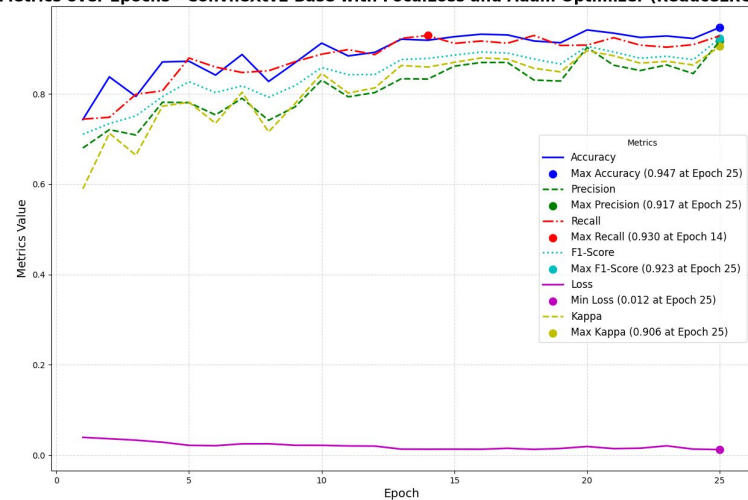
Fine Tuning

Experimental RESULTS (ConvneXt)

Training Metrics over Epochs - ConvneXtv2 Base with FocalLoss and AdamW Optimizer (ReduceLROnPlateau)



Val Metrics over Epochs - ConvneXtv2 Base with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



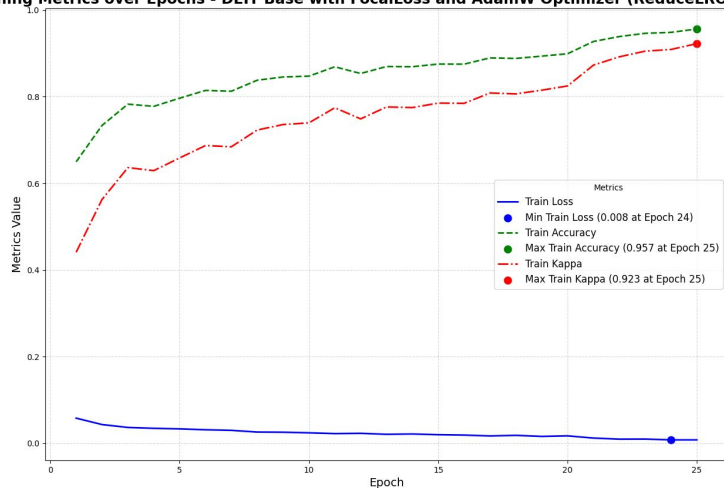
EXPERIMENTS

#Experiment-4

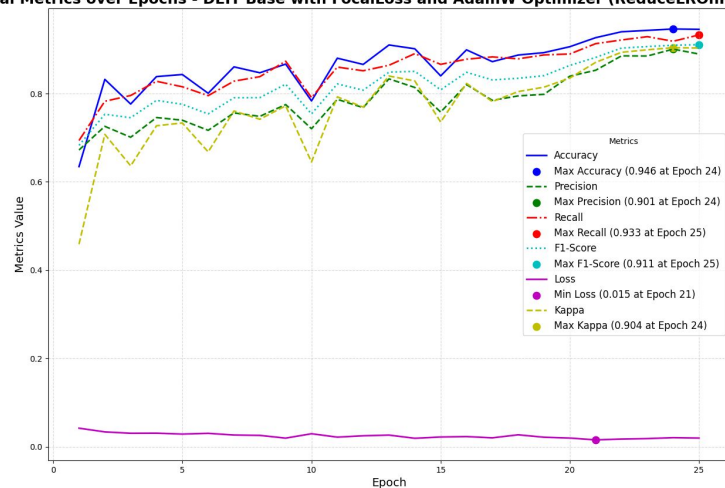
Fine Tuning

Experimental RESULTS (DEiT)

Training Metrics over Epochs - DEiT Base with FocalLoss and AdamW Optimizer (ReduceLROnPlateau)



Val Metrics over Epochs - DEiT Base with FocalLoss and AdamW Optimizer (ReduceLROnPlateau)



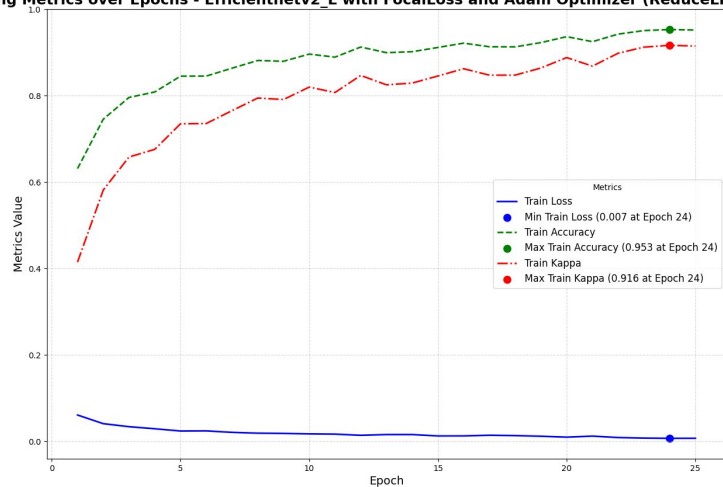
EXPERIMENTS

#Experiment-4

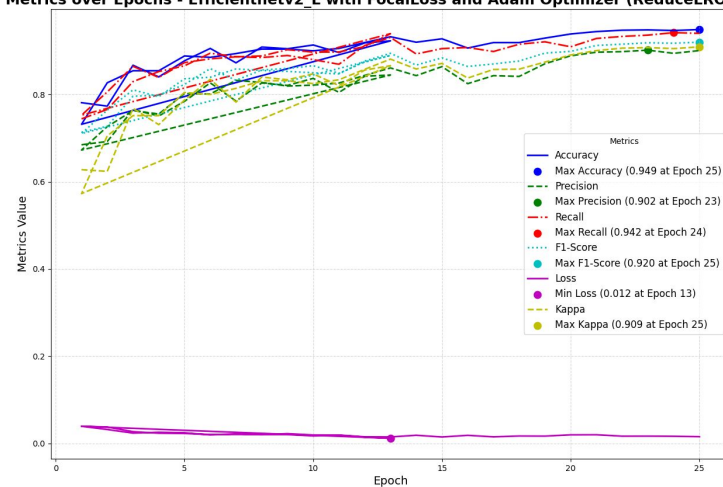
Fine Tuning

Experimental RESULTS (EfficientNet)

Training Metrics over Epochs - Efficientnetv2_L with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



Val Metrics over Epochs - Efficientnetv2_L with FocalLoss and Adam Optimizer (ReduceLROnPlateau)



EXPERIMENTS

#Experiment-5

Ensemble

Experimental RESULTS

Evaluation table challenge 2 (convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9638	0.9349
Hard	0.9591	0.9264

#Experiment-6

Variational Autoencoders

Why VAEs?

- Out of distribution detection.
- Synthetic data generation.

Challenges/Limitations:

- Computational complexity.
- It led us use to use only models that can reconstruct and sample 64x64 pixel images.

EXPERIMENTS

#Experiment-6

Variational Autoencoders

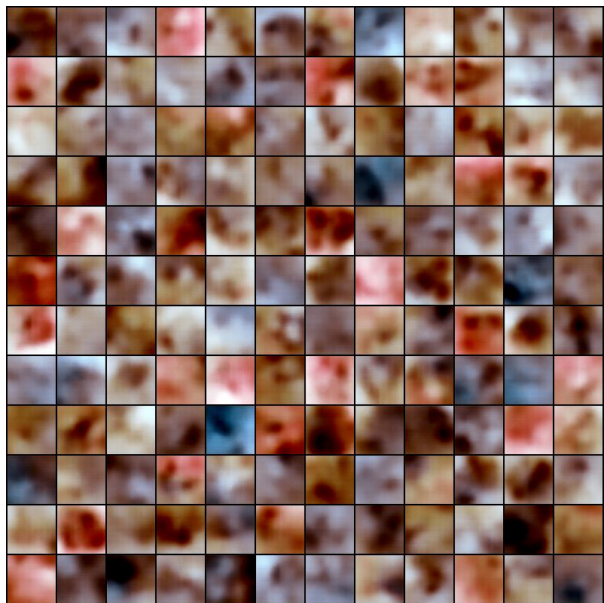
Out of distribution detection

Vanilla VAE	Categorical VAE	Beta VAE
-------------	-----------------	----------

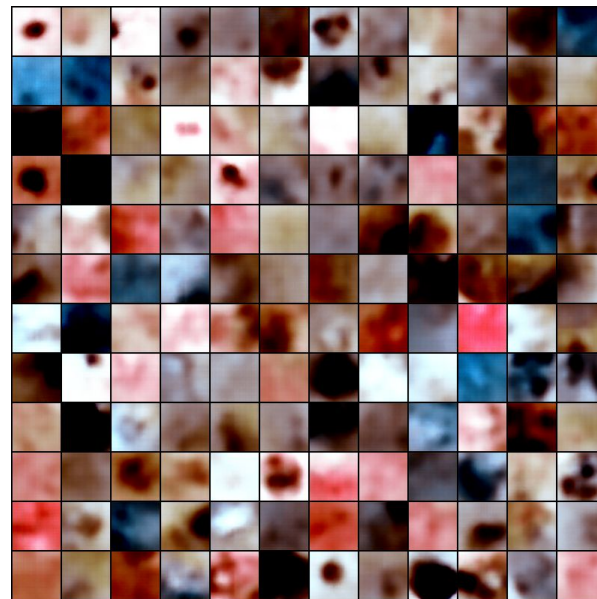
Challenges/Limitations: Reconstruction loss between within-distribution and out of distribution was quite similar.

EXPERIMENTS

#Experiment-6

Variational Autoencoders

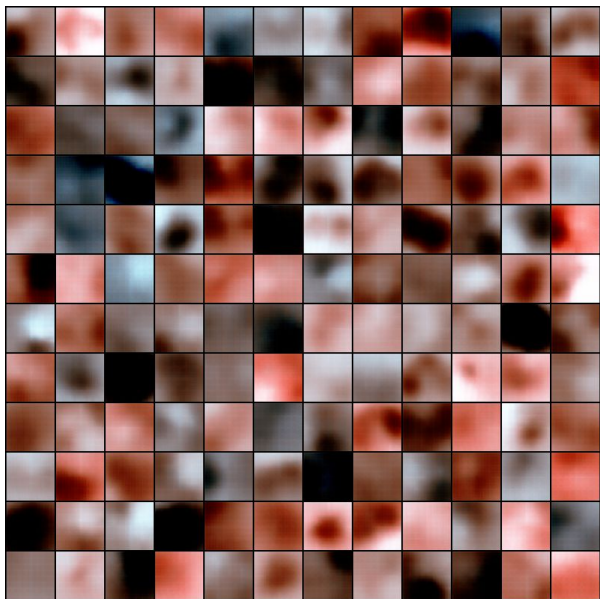
Generated synthetic data

Vanilla VAE

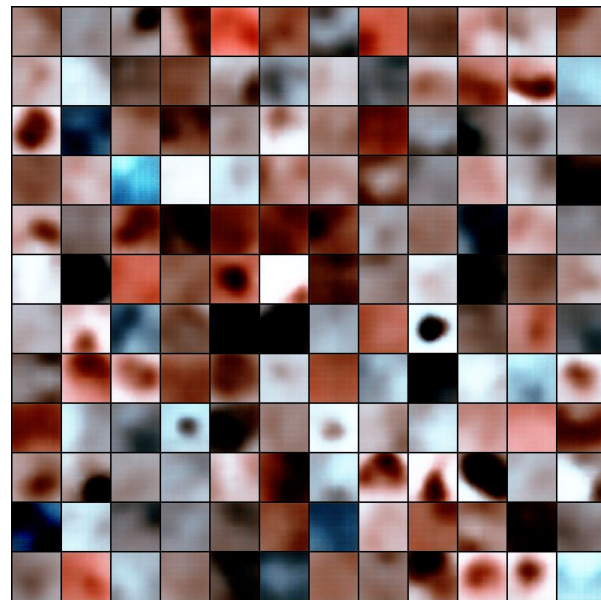
Reconstructed data from validation set

EXPERIMENTS

#Experiment-6

Variational Autoencoders

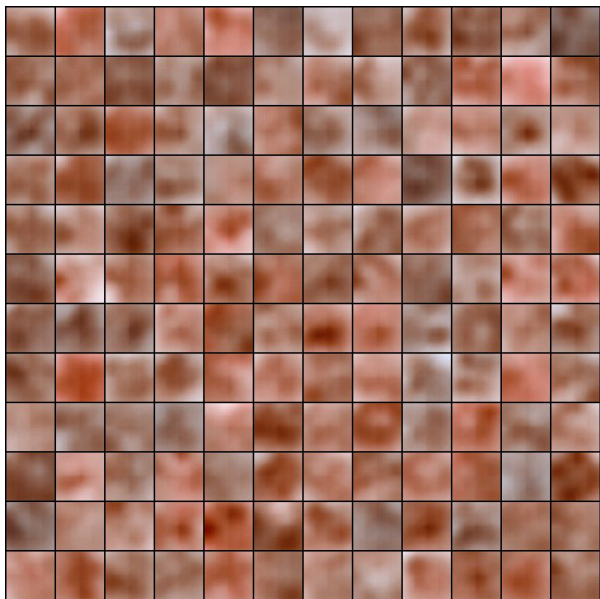
Generated synthetic data

Beta VAE

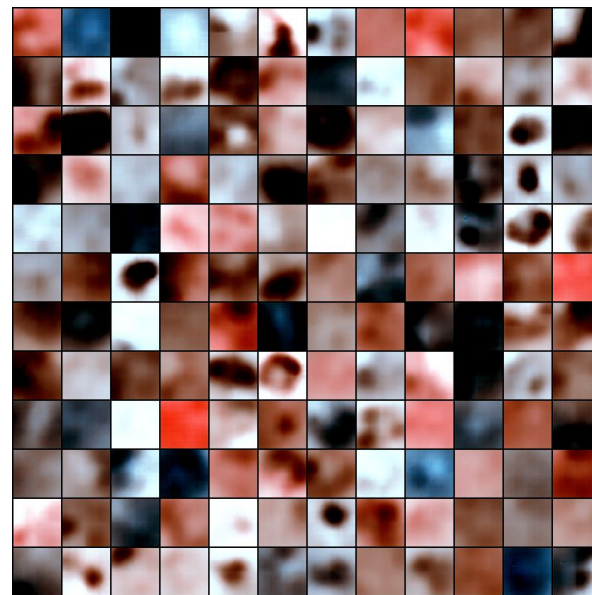
Reconstructed data from validation set

EXPERIMENTS

#Experiment-6

Variational Autoencoders

Generated synthetic data

Categorical VAE

Reconstructed data from validation set

EXPERIMENTS

#Experiment-6

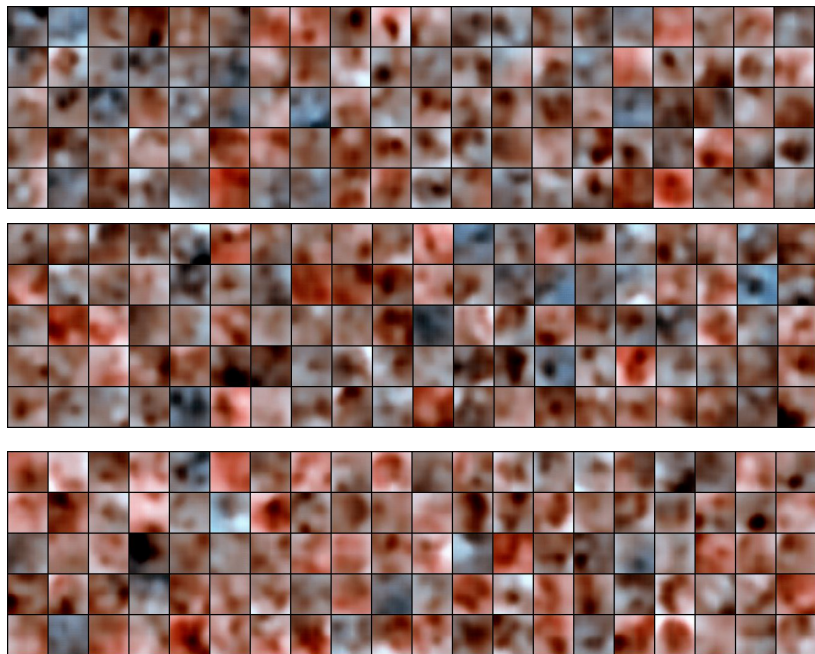
Variational AutoencodersSynthetic Data Generation

Conditional VAE

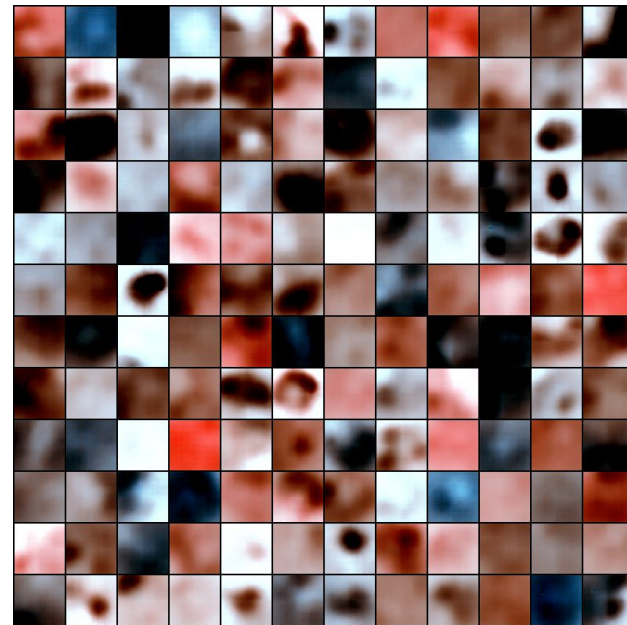
Challenges/Limitations: Since the sampled images are 64x64 pixels, the generated dataset can only be trained with CNNs.

EXPERIMENTS

#Experiment-6

Variational Autoencoders

Generated synthetic data

Conditional VAE

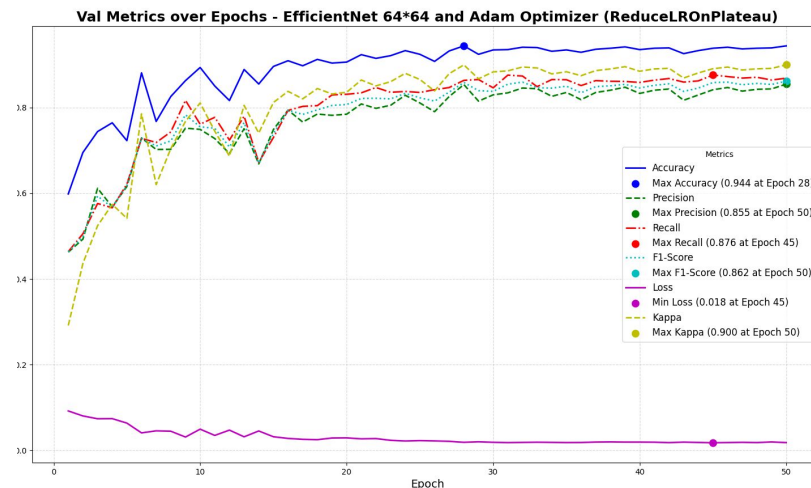
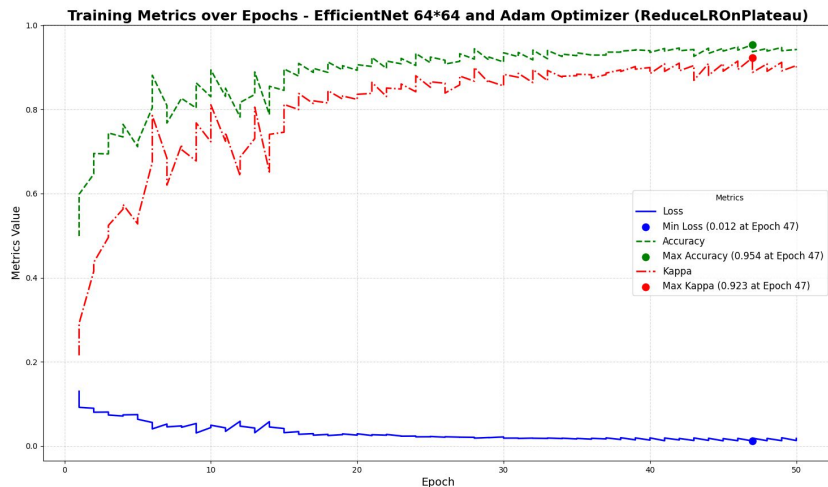
Reconstructed data from validation set

EXPERIMENTS

#Experiment-6

Conditional-VAE (Training with synthetic data)

Experimental RESULTS (EfficientNet)



Comparing with the results obtained using EfficientNET from iteration-4, we observe that the results are quite similar but required more epochs using synthetic data.

Summary of best results



Evaluation table challenge 1 (Convit, convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9294	0.8587
Hard	0.9283	0.8565

Evaluation table challenge 1 (convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9312	0.8624
Hard	0.9320	0.8640

Evaluation table challenge 2 (convnext, deit, efficientnet)

Ensemble Type	Val Accuracy	Val Kappa
Soft	0.9638	0.9349
Hard	0.9591	0.9264

For both challenge 1 and challenge 2 ensemble models gave the best results.

THANK YOU

—